

SIMATS ENGINEERING

ASSESSMENT TOOL 7

COURSE NAME: Operating Systems

COURSE CODE: CSA-04

COURSE OUTCOME COVERED

C03: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:50

Annexure A

EXPERIMENTS

Code of Conduct:

I __HUSSAIN V__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Translation of Logical Address Using Base and Limit Registers

Aim:

To design and implement a program that translates a logical address into a physical address using base and limit registers and identifies invalid addresses.

Theory:

A logical address is generated by the CPU and is relative to the beginning of a process. The base register contains the starting physical address allocated to the process. The limit register specifies the size or maximum valid range of the process. For a logical address L , the address is valid when $0 \leq L < \text{limit}$. If valid, the physical address is calculated as:

Physical Address = Base + Logical Address

If the logical address is greater than or equal to the limit, the operating system rejects the access because it is outside the process's allocated memory.

Algorithm:

1. Read the base register value.
2. Read the limit register value.
3. Read the logical address.
4. Check whether the logical address is less than the limit and non-negative.
5. If valid, calculate physical address = base + logical address.
6. Otherwise, display an invalid-address message.

C Program:

Program:

```
#include <stdio.h>
```

```
int main() {
    int base, limit, logical, physical;

    printf("Enter base address: ");
    scanf("%d", &base);

    printf("Enter limit: ");
    scanf("%d", &limit);

    printf("Enter logical address: ");
    scanf("%d", &logical);

    if (logical >= 0 && logical < limit) {
        physical = base + logical;
        printf("Valid address\n");
        printf("Physical address = %d\n", physical);
    } else {
        printf("Invalid address - Memory access denied\n");
    }

    return 0;
}
```

Sample Output:

Enter base address: 1000

Enter limit: 500

Enter logical address: 250

Valid address

Physical address = 1250

For logical address 600, the output is:

Invalid address - Memory access denied

Result:

The program successfully translates valid logical addresses into physical addresses and rejects addresses outside the process limit.

2. Paging Memory Management Scheme

Aim:

To demonstrate paging memory management by dividing logical memory into fixed-size pages and physical memory into fixed-size frames, then calculating a physical address using a page table.

Theory:

Paging is a non-contiguous memory management technique. Logical memory is divided into equal-sized pages, while physical memory is divided into equal-sized frames. A logical address contains two parts: a page number and an offset. The page table gives the frame number corresponding to each page.

If the page size is P and the logical address is represented by page number p and offset d, the physical address is:

Physical Address = Frame Number × Page Size + Offset

The offset must be smaller than the page size. Paging eliminates the need for a process to occupy one continuous region of physical memory.

Algorithm:

1. Read the page size.
2. Enter the number of pages and their corresponding frame numbers.
3. Read the page number and offset.
4. Verify that the page number is valid and the offset is less than the page size.
5. Find the frame number from the page table.
6. Calculate physical address = frame × page size + offset.
7. Display the translated physical address.

C Program:

Program:

```
#include <stdio.h>
```

```
int main() {
    int pageSize, pages, page, offset, frame[20], physical;

    printf("Enter page size: ");
    scanf("%d", &pageSize);

    printf("Enter number of pages: ");
    scanf("%d", &pages);

    printf("Enter frame number for each page:\n");
    for (int i = 0; i < pages; i++) {
        printf("Page %d: ", i);
        scanf("%d", &frame[i]);
    }

    printf("Enter page number: ");
    scanf("%d", &page);

    printf("Enter offset: ");
    scanf("%d", &offset);
```

```
if (page >= 0 && page < pages && offset >= 0 && offset < pageSize) {  
    physical = frame[page] * pageSize + offset;  
    printf("Page %d is mapped to Frame %d\n", page, frame[page]);  
    printf("Physical address = %d\n", physical);  
} else {  
    printf("Invalid page number or offset\n");  
}  
  
    return 0;  
}
```

Sample Output:

Enter page size: 100

Enter number of pages: 4

Enter frame number for each page:

Page 0: 3

Page 1: 1

Page 2: 4

Page 3: 2

Enter page number: 2

Enter offset: 25

Page 2 is mapped to Frame 4

Physical address = 425

Result:

The program successfully demonstrates paging by using a page table to convert a page number and offset into a physical address.

3. Simulation of a Page Table

Aim:

To develop a program that accepts page table entries and logical addresses, maps logical page numbers to physical frame numbers, and displays the resulting physical addresses.

Theory:

A page table is maintained by the operating system for each process. Each page table entry contains the frame number in which a particular page is stored. When a CPU generates a logical address, the page number is used to index the page table. The corresponding frame number is then combined with the offset to produce the physical address.

For page size P:

Physical Address = Frame Number $\times$ P + Offset

This experiment clearly shows the relationship between page numbers and frame numbers.

Algorithm:

1. Read page size and number of pages.
2. Store the frame number for every page in the page table.
3. Read a logical page number and offset.
4. Search the page table using the page number.
5. Display the page-to-frame mapping.
6. Calculate and display the physical address.
7. Report an invalid page or offset when necessary.

C Program:

Program:

```
#include <stdio.h>
```

```
int main() {
    int n, size, page, offset, frame[20];

    printf("Enter page size: ");
    scanf("%d", &size);

    printf("Enter number of pages: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter frame for page %d: ", i);
        scanf("%d", &frame[i]);
    }

    printf("\nPage Table:\n");
    printf("Page\tFrame\n");
    for (int i = 0; i < n; i++)
        printf("%d\t%d\n", i, frame[i]);

    printf("Enter page number: ");
    scanf("%d", &page);
```

```

printf("Enter offset: ");
scanf("%d", &offset);

if (page >= 0 && page < n && offset >= 0 && offset < size) {
    printf("Page %d -> Frame %d\n", page, frame[page]);
    printf("Physical Address = %d\n",
        frame[page] * size + offset);
} else {
    printf("Invalid logical address\n");
}

return 0;
}

```

Sample Output:

```

Enter page size: 100
Enter number of pages: 3
Enter frame for page 0: 5
Enter frame for page 1: 2
Enter frame for page 2: 7
Page Table:
Page  Frame
0      5
1      2
2      7

```

```

Enter page number: 1
Enter offset: 40
Page 1 -> Frame 2
Physical Address = 240

```

Result:

The page table successfully maps the logical page number to its corresponding physical frame and generates the physical address.

4. Simulation of a Translation Lookaside Buffer (TLB)

Aim:

To simulate a Translation Lookaside Buffer and demonstrate how it reduces memory access time by comparing access with TLB and without TLB.

Theory:

A Translation Lookaside Buffer is a small, fast associative memory that stores recently used page-table entries. Without a TLB, accessing a memory location generally requires first accessing the page table and then accessing the actual memory. With a TLB hit, the frame number can be obtained quickly, reducing the average memory access time.

Let:

TLB lookup time = t

Memory access time = m

TLB hit ratio = h

A simplified effective access time is:

$$EAT = h(t + m) + (1 - h)(t + 2m)$$

Without TLB, the simplified access time is:

$$EAT \text{ without TLB} = 2m$$

The exact formula can vary depending on the memory system, but this experiment uses the above common model.

Algorithm:

1. Read TLB lookup time, memory access time, and TLB hit ratio.
2. Calculate the hit time and miss time.
3. Calculate effective access time using TLB.
4. Calculate access time without TLB.
5. Compare both values.
6. Display the improvement.

C Program:

Program:

```
#include <stdio.h>
```

```
int main() {
    float t, m, h, eat, without;

    printf("Enter TLB lookup time: ");
    scanf("%f", &t);

    printf("Enter memory access time: ");
    scanf("%f", &m);

    printf("Enter TLB hit ratio (0 to 1): ");
    scanf("%f", &h);

    eat = h * (t + m) + (1 - h) * (t + 2 * m);
    without = 2 * m;

    printf("Effective access time with TLB = %.2f\n", eat);
    printf("Access time without TLB = %.2f\n", without);
}
```



```
    if (eat < without)
        printf("TLB improves memory access time.\n");

    return 0;
}
```

Sample Output:

```
Enter TLB lookup time: 10
Enter memory access time: 100
Enter TLB hit ratio (0 to 1): 0.8
Effective access time with TLB = 130.00
Access time without TLB = 200.00
TLB improves memory access time.
```

Result:

The program demonstrates that a high TLB hit ratio reduces the average memory access time because frequently used page-table entries are found in the fast TLB.

5. Memory Protection Using Base and Limit Registers

Aim:

To implement memory protection using base and limit registers and prevent unauthorized access to memory locations outside the process's allocated range.

Theory:

Memory protection ensures that one process cannot access memory belonging to another process or to the operating system. The base register stores the starting physical address of the process, while the limit register stores the size of the memory region allocated to the process.

For a logical address L, access is permitted only when:

$$0 \leq L < \text{limit}$$

If the condition is satisfied:

$$\text{Physical Address} = \text{Base} + L$$

Otherwise, the operating system generates a protection violation and denies the memory access. This mechanism is simple and provides hardware-supported protection for contiguous memory allocation.

Algorithm:

1. Read the base address.
2. Read the limit value.
3. Read the logical address requested by the process.
4. Check whether the logical address is within the limit.
5. If valid, calculate the physical address.
6. If invalid, display a protection violation message.
7. Do not allow the unauthorized memory access.

C Program:

Program:

```
#include <stdio.h>
```

```
int main() {
    int base, limit, logical, physical;

    printf("Enter base register: ");
    scanf("%d", &base);

    printf("Enter limit register: ");
    scanf("%d", &limit);

    printf("Enter requested logical address: ");
    scanf("%d", &logical);

    if (logical >= 0 && logical < limit) {
        physical = base + logical;
        printf("Access Allowed\n");
        printf("Physical Address = %d\n", physical);
    } else {
        printf("Access Denied!\n");
        printf("Protection Violation: Unauthorized memory access.\n");
    }
}
```

```
    return 0;  
}
```

Sample Output:

Enter base register: 5000

Enter limit register: 1000

Enter requested logical address: 700

Access Allowed

Physical Address = 5700

For logical address 1200:

Access Denied!

Protection Violation: Unauthorized memory access.

Result:

The program successfully verifies logical addresses against the limit register and prevents access to memory outside the process's permitted region.

RUBRICS FOR CALCULATION:

Criteria	Weight age (Marks)	Level 4 - Exemplary (Full Marks)	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Conceptual Understand ing	35 Marks	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25 Marks	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Applicatio n of Knowledge	15 Marks	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completen ess of Response	15 Marks	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentati on	10 Marks	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand